Exercise 1

Nico Reiterer

November 2026

Exercise 1 - Variance after a change of origin:

Tasks 1 - 3

```
set.seed(1)
z <- rnorm(100000, mean = 0, sd = 3)
offsets <- c(0, 1e4, 1e8, 1e12, 1e14)

# define the algorithms
## two-pass variance
two_pass <- function(x) {
  x_bar <- mean(x)
  return(sum((x - x_bar)^2) / (length(x) - 1))
}

## one-pass variance
one_pass <- function(x) {
  n <- length(x)
  sum_x  <- sum(x)
  sum_x2 <- sum(x^2)
  (sum_x2 - sum_x^2 / n) / (n - 1)
}

# prep the df for caluclating later
results <- data.frame(
  a = numeric(),
  `var(x)` = numeric(),
  `two-pass` = numeric(),
  `one-pass` = numeric(),
  check.names = FALSE
)

for (a in offsets){
  x <- a + z

  new_row <- data.frame(
    a = a,
    `var(x)` = var(x),
    `two-pass`= two_pass(x),
    `one-pass` = one_pass(x),
    check.names = FALSE)
```

```
  # Append the row
  results <- rbind(results, new_row)
}

results
```

```
##       a   var(x) two-pass      one-pass
## 1 0e+00 9.063530 9.063530  9.063530e+00
## 2 1e+04 9.063530 9.063530  9.063529e+00
## 3 1e+08 9.063530 9.063530  4.325419e+01
## 4 1e+12 9.063529 9.063529  1.329983e+11
## 5 1e+14 9.063590 9.063590 -8.195913e+15
```

4. Compare each result with var(z). At approximately what size of a does the one-pass formula begin to lose useful accuracy

Down below you can see that one-pass at appr. size a = 1e8 begins to lose useful accuracy.

```
true_var <- var(z)
methods  <- c("var(x)", "two-pass", "one-pass")
rel_err  <- sapply(methods, function(m) abs(results[[m]] - true_var) / true_var)

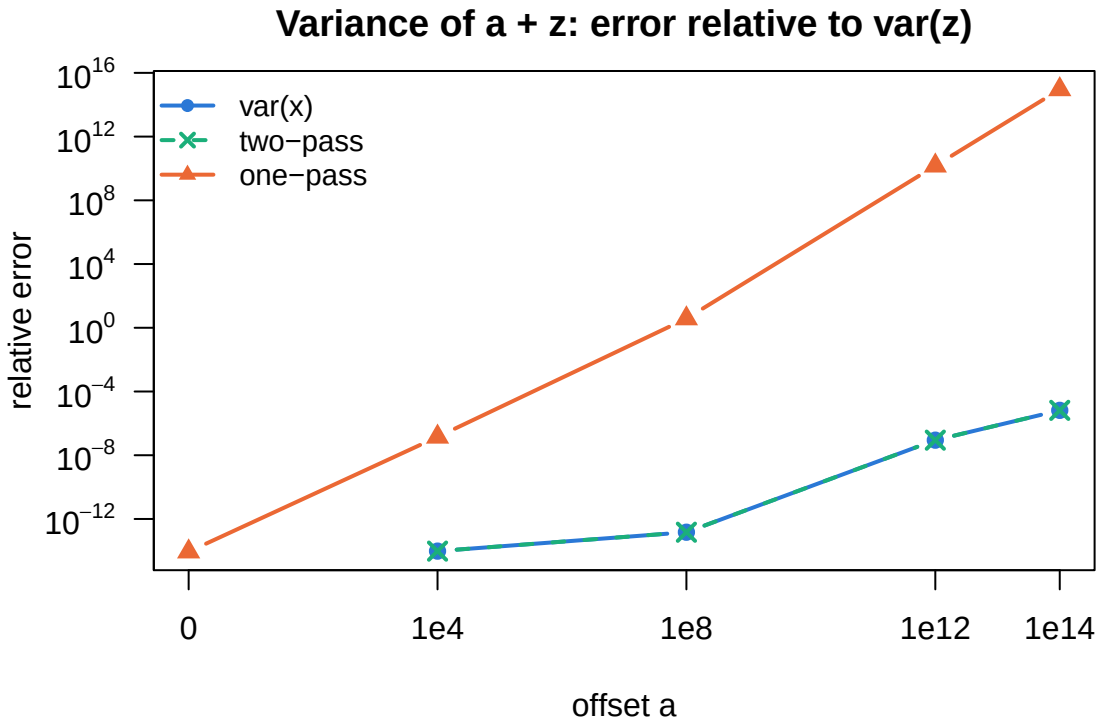

cols <- c("#2a78d6", "#1baf7a", "#eb6834")
pchs <- c(16, 4, 17)
ltys <- c(1, 2, 1)
x_at <- ifelse(offsets == 0, 1, offsets)      # a = 0 shown at 10^0 on the log axis

par(mar = c(4.5, 4.5, 2.5, 1), las = 1)
matplot(x_at, rel_err, log = "xy", type = "b",
        col = cols, pch = pchs, lty = ltys, lwd = 2, cex = 1.2,
        xaxt = "n", yaxt = "n",
        xlab = "offset a", ylab = "relative error",
        main = "Variance of a + z: error relative to var(z)")
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log = log, recycle = TRUE): 2 y
## values <= 0 omitted from logarithmic plot
```

```
## Warning in xy.coords(x, y, xlabel, ylabel, log): 1 y value <= 0 omitted from
## logarithmic plot
```

```
axis(1, at = x_at, labels = c("0", "1e4", "1e8", "1e12", "1e14"))
y_ticks <- 10^seq(-16, 16, by = 4)
axis(2, at = y_ticks, labels = parse(text = paste0("10^", log10(y_ticks))))
legend("topleft", legend = methods, col = cols, pch = pchs, lty = ltys,
       lwd = 2, bty = "n", cex = 0.9)
```



5. Explain why adding a constant leaves the variance unchanged mathematically but can still affect a particular numerical algorithm.

Because a constant is constant essentially, it "changes the location but not the variance". You could also argue with the following where Y would be the constant.

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X,Y).$$

Y is constant, hence Var(Y) is zero. Further, there is no covariance/dependence between an added constant Y and X, hence COV(X,Y) must be 0 as well.

But okay, why does it affect us here then? Because of machine precision: At a=1e8 we have for the one-pass a square of x_i which leads to us the limit of double precision (1e16). Then the subtraction of such huge numbers leads to rounding errors that do not represent anymore the actual calculation. Two-pass is not as affected as we only square the the difference between the mean and the actual values. Still at big enough a, we can run into the limit described above!

6. Suppose you receive measurements in nanometres with values around 10^{12}, but the scientifically relevant variation is only a few units. Would you preprocess the data before computing descriptive statistics? If yes, what transformation would you use and why?

As we have just shown, shifting by a constant is not influencing the statistics itself. Hence to have a proper view on the relativeness of the descriptive statistics, I would subtract by this common value of 10^{12}. Still also things like the mean are only shifted then and can still be reproduced by readding that subtrahend at the end.

Exercise 2 - A probability that disappears

1. Compute the upper-tail probability in each of the following ways:

```
x_tail <- c(5, 8, 10, 20, 30, 40)
```

```r
data.frame(
  x = x_tail,
  pnorm = pnorm(x_tail),
  one_minus_pnorm = 1 - pnorm(x_tail),
  pnorm_upper_tail = pnorm(x_tail, lower.tail = FALSE),
  log_upper_tail = pnorm(x_tail, lower.tail = FALSE, log.p = TRUE)
)
```

```
##    x     pnorm one_minus_pnorm pnorm_upper_tail log_upper_tail
## 1  5 0.9999997    2.866516e-07     2.866516e-07      -15.06500
## 2  8 1.0000000    6.661338e-16     6.220961e-16      -35.01344
## 3 10 1.0000000    0.000000e+00     7.619853e-24      -53.23129
## 4 20 1.0000000    0.000000e+00     2.753624e-89     -203.91716
## 5 30 1.0000000    0.000000e+00    4.906714e-198     -454.32124
## 6 40 1.0000000    0.000000e+00     0.000000e+00     -804.60844
```

2. and 3. Identify where the first expression begins to give a misleading answer and explain why 1 - pnorm(x) can fail even when pnorm(x) itself is accurately compute

At x = 10 already because the machine precision doesn't allow for more precision here when we do the subtraction of 1 - pnorm. This is just rounded to 1 or 0 respectively. Using pnorm with lower.tail = FALSE computes the upper tail directly instead of doing the subtraction, which avoids the rounding issue when subtracting.

4. For each x, recover a probability from the log-probability using exp(). At what point does even that conversion become unhelpful

```r
x_tail <- c(5, 8, 10, 20, 30, 35, 36, 37, 38, 39,40)

data.frame(
  x = x_tail,
  pnorm = pnorm(x_tail),
  pnorm_upper_tail = pnorm(x_tail, lower.tail = FALSE),
  log_upper_tail = pnorm(x_tail, lower.tail = FALSE, log.p = TRUE),
  `exp(log_upper_tail)` = exp(pnorm(x_tail, lower.tail = FALSE, log.p = TRUE)),
  check.names = FALSE

)
```

```
##     x     pnorm pnorm_upper_tail log_upper_tail exp(log_upper_tail)
## 1   5 0.9999997     2.866516e-07      -15.06500        2.866516e-07
## 2   8 1.0000000     6.220961e-16      -35.01344        6.220961e-16
## 3  10 1.0000000     7.619853e-24      -53.23129        7.619853e-24
## 4  20 1.0000000     2.753624e-89     -203.91716        2.753624e-89
## 5  30 1.0000000    4.906714e-198     -454.32124       4.906714e-198
## 6  35 1.0000000    1.124911e-268     -616.97510       1.124911e-268
## 7  36 1.0000000    4.182624e-284     -652.50323       4.182624e-284
## 8  37 1.0000000    5.725571e-300     -689.03059       5.725571e-300
## 9  38 1.0000000     0.000000e+00     -726.55722       2.885428e-316
## 10 39 1.0000000     0.000000e+00     -765.08316        0.000000e+00
## 11 40 1.0000000     0.000000e+00     -804.60844        0.000000e+00
```

So as can be seen quite unhelpful at the stage of x = 39 when it turns 0 again.

5. Imagine a multiple-testing problem with a test statistic corresponding to Z = 9. You need the p-value to compare it with a very stringent threshold. Which representation of the probability would you carry through the computation, and why?

I would go for the log-probability representation as this is exactly a situation where we would need precicion (as indicated by the "stringent threshold"). If we were to report on 1-Z via 1-pnorm(9) we would already get 0, which is well quite wrong here.

Exercise 3 - Comparing two models without computing their likelihood

Tasks 1-3

```
set.seed(2)
y <- rbinom(5000, size = 1, prob = 0.03)
p1 <- 0.03
p2 <- 0.035

L <- function(p, y) {
  result <- prod(p**y * (1 - p)**(1-y))
  return(result)
}

l <- function(p, y) {
  return(sum(y * log(p) + (1 - y) * log(1 - p)))
}


data.frame(
  "$p_1$" = c(p1, p2),
  "$L(p)$" = c(L(p1, y), L(p2, y)),
  "$l(p)$" = c(l(p1, y), l(p2, y)),
  check.names = FALSE)
```

```
##   $p_1$ $L(p)$    $l(p)$
## 1 0.030      0 -750.1850
## 2 0.035      0 -748.6221
```

We can see the theory from the lecture confirmed: *The product underflows to zero, but the log-likelihood is still finite.*

4. Compute the log-likelihood difference l(p1) - l(p2). Explain why this quantity can be more useful than either likelihood itself.

```
print(l(p1, y))
```

```
## [1] -750.185
```

```
print(l(p2, y))
```

```
## [1] -748.6221
```

```
l(p1, y) - l(p2, y)
```

```
## [1] -1.562957
```

A positive value of the difference means the data is in favor of p_1 more than p_2. A negative value means the opposite. The magnitude says how strong that support is on the log scale and it is also numerically save (whole idea of this exercise I suppose). So the subtraction should never really have an underflow as opposed to the 0 rounding for the L(p) function!

5. Compute the likelihood ratio L(p1)/ L(p2) in a numerically stable way using the log-likelihoods.

Easily via log and then exp again:

$$log(L(p_1)/L(p_2)) = log(L(p_1)) - log(L(p_2)) = l(p_1, y) - l(p_2, y)$$

$$L(p_1)/L(p_2) = exp(l(p_1, y) - l(p_2, y))$$

```
log_LR <- l(p1, y) - l(p2, y)
LR <- exp(log_LR)
print(LR)
```

```
## [1] 0.2095156
```

```
print(L(p1, y) / L(p2, y))   # For comparison, the direct way fails: division by zero :(
```

```
## [1] NaN
```

6. Why is this issue especially important in Bayesian computation, maximum likelihood estimation, and sequential analysis?

- Bayesian: Because e.g. the posterior directly depends on the likelihood. Meaning if we do not take case of the underflow or the such, then the calculation could become wrong along the way
- Maximum Likelihood Estimation: Essentially the same reason, the estimation needs the Likelihood function to be unequal zero (or at least not zero because of technical reasons!) in order to find a fitting distribution for the data
- Sequential Analysis: Not done that yet actually, but as far as I can gather from Wikipedia this directly depends on the likelihoods again: $S_i = S_{i-1} + log\lambda_i$ with the lambda being the ratio of Likelihoods before and after. Here we could easily over or underflow again in each step, rendering the result useless!

Exercise 4 - Diagnosing a numerically suspicious covariance matrixo

Tasks 1 to 6

```
# Consider two variables that are almost linear copies of each other:
set.seed(3)
n   <- 5000
u   <- rnorm(n)
x1  <- 1e8 + u
x2  <- 1e8 + u + rnorm(n, sd = 1e-4)
X   <- cbind(x1, x2)

# 1. Covariance
covar_matrix <- cov(X)
print(covar_matrix)
```

```
##          x1       x2
## x1 1.012977 1.012978
## x2 1.012978 1.012979
```

```
# 2. Correlation bet. x1 and x2
correlation  <- cor(x1, x2)
print(correlation, digits=16)
```

```
## [1] 0.9999999950714031
```

```
# 3. Eigenvals of the covariance matrix
eigenvals <- eigen(covar_matrix)

# 4. Covariance with one-identity
cov_simple <- function(a, b) {
  n <- length(a)
  cat("E(a, b) = ", mean(a*b), "\n") # just to show again machine precision!
  (mean(a * b) - mean(a) * mean(b)) * n / (n - 1)
}

cov_alternative <- matrix(c(cov_simple(x1, x1), cov_simple(x1, x2),
                            cov_simple(x2, x1), cov_simple(x2, x2)),
                          nrow = 2)
```

```
## E(a, b) =  1e+16
## E(a, b) =  1e+16
## E(a, b) =  1e+16
## E(a, b) =  1e+16
```

```
print("cov(x)")
```

```
## [1] "cov(x)"
```

```
print(covar_matrix)
```

```
##          x1       x2
## x1 1.012977 1.012978
## x2 1.012978 1.012979
```

```
print(eigenvals)
```

```
## eigen() decomposition
## $values
## [1] 2.025956e+00 4.992561e-09
##
## $vectors
##           [,1]       [,2]
## [1,] 0.7071066 -0.7071070
## [2,] 0.7071070  0.7071066
```

```
print("")
```

```
## [1] ""
```

```
print("one-pass")
```

```
## [1] "one-pass"
```

```
print(cov_alternative)
```

```
##        [,1]   [,2]
## [1,] 2.0004 2.0004
## [2,] 2.0004 2.0004
```

```
print(eigen(cov_alternative))
```

```
## eigen() decomposition
## $values
## [1] 4.0008 0.0000
##
## $vectors
##           [,1]       [,2]
## [1,] 0.7071068 -0.7071068
## [2,] 0.7071068  0.7071068
```

```
# Repeat the calculation after centring both variables first
Xc  <- scale(X, center = TRUE, scale = FALSE)
x1c <- scale(x1, center = TRUE, scale = FALSE)
x2c <- scale(x2, center = TRUE, scale = FALSE)


covar_matrix_c  <- cov(Xc)
correlation_c   <- cor(x1c, x2c)
eigenvals_c     <- eigen(covar_matrix)

# Cov(X, Y) = E(XY) - E(X)E(Y)
cov_alternative_c <- matrix(c(cov_simple(x1c, x1c), cov_simple(x1c, x2c),
                              cov_simple(x2c, x1c), cov_simple(x2c, x2c)),
                              nrow = 2)
```

```
## E(a, b) =  1.012775
## E(a, b) =  1.012775
## E(a, b) =  1.012775
## E(a, b) =  1.012776
```

```
print("cov(x)")
```

```
## [1] "cov(x)"
```

```
print(covar_matrix_c)
```

```
##          x1       x2
## x1 1.012977 1.012978
## x2 1.012978 1.012979
```

```
print(eigenvals_c)
```

```
## eigen() decomposition
## $values
## [1] 2.025956e+00 4.992561e-09
##
## $vectors
##           [,1]       [,2]
## [1,] 0.7071066 -0.7071070
## [2,] 0.7071070  0.7071066
```

```
print("")
```

```
## [1] ""
```

```
print("one-pass")
```

```
## [1] "one-pass"
```

```
print(cov_alternative_c)
```

```
##          [,1]     [,2]
## [1,] 1.012977 1.012978
## [2,] 1.012978 1.012979
```

```
print(eigen(cov_alternative_c))
```

```
## eigen() decomposition
## $values
## [1] 2.025956e+00 4.992558e-09
##
## $vectors
##           [,1]       [,2]
## [1,] 0.7071066 -0.7071070
## [2,] 0.7071070  0.7071066
```

7. A statistical procedure requires inversion of the covariance matrix. What warning signs in this example would make you cautious about doing that directly? Suggest at least two safer approaches.

- First thing for sure is the issue with eigenvalue = 0, as this means a matrix would not even be invertible
- Also if we see the correlation is really close to 1 (which it is here, irrespective of the floating issues) then we might also run into a singular matrix, since the columns seem to be quite heavily related.

In any way for these cases we

(a) Should center the data as we did above (and is done by the native cov() function) so we can still work with it in case it is really a non-singular matrix (and not only because of machine precision)

(b) Could also "fall back" to using SVD where "Small singular values directly reveal near rank-deficiency"

Exercise 5 - Whitening is not unique

1. Generate the random data

```
set.seed(3)
n   <- 1000
dims <- 5

# Independent std normal variables
independent <- matrix(rnorm(n * dims), nrow = n, ncol = dims)
cov_ind <- cov(independent)

suitable_matrix <- matrix(c(
  1.0000, 1.200, 0.18, 0.324, 0.1296,
  1.2000, 4.000, 0.60, 1.080, 0.4320,
  0.1800, 0.600, 0.25, 0.450, 0.1800,
  0.3240, 1.080, 0.45, 2.250, 0.9000,
  0.1296, 0.432, 0.18, 0.900, 1.0000
), nrow = 5, byrow = TRUE)
```

```r
# to show it is all > 0, hence positive definite
print(eigen(suitable_matrix)$values)
```

```
## [1] 5.1731424 2.1076658 0.5794569 0.5261850 0.1135499
```

So, we are supposed to construct this normally distributed matrix *independent (E)* and then multiply it with something else that gives us a non-diagonal covariance matrix. For this we build the *suitable_matrix(A)* (just some random non-diagonal covariance matrix). In order for the new big sample to have the same we multiply them together and it follows that the new covariance will be what we just build

$$Cov(EA) = A^T Cov(E) A \approx A^T I A = A^T A$$

```r
# The Cholesky Decomposition - Compute the Cholesky factorization of a real symmetric positive-definite
A <- chol(suitable_matrix)


X <- independent %*% A

# efficiently for row-wise observations as per chapter "Cross-product also appear in covariance"
Xc <- scale(X, center = TRUE, scale = FALSE)
S <- crossprod(Xc) / (nrow(Xc) - 1)

print(S)
```

```
##            [,1]      [,2]      [,3]      [,4]       [,5]
## [1,] 0.99615454 1.1507763 0.1787771 0.2849052 0.08154955
## [2,] 1.15077628 3.8730603 0.5641581 0.9960214 0.39627759
## [3,] 0.17877706 0.5641581 0.2448059 0.4643973 0.17493224
## [4,] 0.28490520 0.9960214 0.4643973 2.3567243 0.83433491
## [5,] 0.08154955 0.3962776 0.1749322 0.8343349 0.95190685
```

```r
print(cov(Xc))
```

```
##            [,1]      [,2]      [,3]      [,4]       [,5]
## [1,] 0.99615454 1.1507763 0.1787771 0.2849052 0.08154955
## [2,] 1.15077628 3.8730603 0.5641581 0.9960214 0.39627759
## [3,] 0.17877706 0.5641581 0.2448059 0.4643973 0.17493224
## [4,] 0.28490520 0.9960214 0.4643973 2.3567243 0.83433491
## [5,] 0.08154955 0.3962776 0.1749322 0.8343349 0.95190685
```

2. Compute the eigendecomposition + PCA whitening matrix

```r
S_eigen <- eigen(S)
lambdas <- S_eigen$values
Qs      <- S_eigen$vectors

W_PCA <- diag(lambdas**(-1/2)) %*% t(Qs)

W_PCA %*% S %*% t(W_PCA) # confirming the Identity
```

```
##               [,1]          [,2]          [,3]          [,4]          [,5]
## [1,]  1.000000e+00 -9.474631e-17  1.082842e-15 -7.252329e-16 -2.384586e-16
## [2,] -1.489355e-16  1.000000e+00  5.636539e-17 -1.174146e-16  1.013314e-15
## [3,]  1.009281e-15  5.272489e-17  1.000000e+00  1.300587e-15 -6.994002e-16
```

```
## [4,] -6.421831e-16  3.288181e-17  1.327569e-15  1.000000e+00 -6.584309e-16
## [5,] -1.816909e-16  9.393377e-16 -7.455753e-16 -6.169810e-16  1.000000e+00
```

```
# applying it now finally
Z_PCA <- Xc %*% t(W_PCA)
cov(Z_PCA) # nice also identity here as wanted!
```

```
##               [,1]          [,2]          [,3]          [,4]          [,5]
## [1,]  1.000000e+00  1.823700e-16  1.449515e-15 -1.151009e-15 -2.577184e-16
## [2,]  1.823700e-16  1.000000e+00 -1.260181e-15 -3.391931e-16  2.883301e-15
## [3,]  1.449515e-15 -1.260181e-15  1.000000e+00  1.206137e-15 -2.500127e-15
## [4,] -1.151009e-15 -3.391931e-16  1.206137e-15  1.000000e+00 -2.063373e-15
## [5,] -2.577184e-16  2.883301e-15 -2.500127e-15 -2.063373e-15  1.000000e+00
```

3. Construct the ZCA whitening matrix

```
W_ZCA <- Qs %*% diag(lambdas**(-1/2)) %*% t(Qs)
Z_ZCA <- Xc %*% t(W_ZCA)
cov(Z_ZCA) # nice also identity here as wanted!
```

```
##               [,1]          [,2]          [,3]          [,4]          [,5]
## [1,]  1.000000e+00 -1.199519e-15 -4.468398e-16  8.294045e-16 -1.200338e-15
## [2,] -1.199519e-15  1.000000e+00  2.289793e-15  1.418966e-16  8.290276e-16
## [3,] -4.468398e-16  2.289793e-15  1.000000e+00 -2.553794e-15  1.803432e-15
## [4,]  8.294045e-16  1.418966e-16 -2.553794e-15  1.000000e+00 -1.889751e-15
## [5,] -1.200338e-15  8.290276e-16  1.803432e-15 -1.889751e-15  1.000000e+00
```

```
# Interestingly, this ZCA is essentially S^-1/2 if we take a look at how it is being constructed.
# It is essentially the eigendecomposition of S, just with ^-1/2 scaled eigenvalues...
# W_ZCA is then also just a rotation of W_PCA as we have to do Q * W_PCA to get to ZCA...
```

4. Compute a Cholesky factorization

```
# attention: chol(S) returns R^TR not LL^T like in the exercise sheet
R <- chol(S)
L <- t(R)
L_inv <- solve(L)

Z_chol <- Xc %*% t(L_inv)
print(cov(Z_chol))
```

```
##               [,1]          [,2]          [,3]          [,4]          [,5]
## [1,]  1.000000e+00 -7.182554e-16  2.459939e-16  4.642044e-16 -2.528286e-18
## [2,] -7.182554e-16  1.000000e+00  3.933568e-16 -1.008036e-15  7.966878e-17
## [3,]  2.459939e-16  3.933568e-16  1.000000e+00 -5.837700e-16  8.216859e-16
## [4,]  4.642044e-16 -1.008036e-15 -5.837700e-16  1.000000e+00 -1.866583e-15
## [5,] -2.528286e-18  7.966878e-17  8.216859e-16 -1.866583e-15  1.000000e+00
```

```
L_inv %*% S %*% t(L_inv) # confirming the Identity, interesting
```

```
##               [,1]          [,2]          [,3]          [,4]          [,5]
## [1,]  1.000000e+00  1.474576e-16  3.118432e-17 -4.241838e-17 -1.860952e-17
## [2,]  1.830744e-16  1.000000e+00 -1.040522e-16 -4.459540e-17  2.310472e-17
## [3,] -1.092105e-16 -1.825495e-16  1.000000e+00 -1.766484e-16  9.287319e-17
## [4,]  1.701502e-17 -3.484846e-18 -7.741755e-17  1.000000e+00 -3.719681e-17
## [5,] -2.376701e-17  3.333434e-19  7.621308e-17 -6.253623e-17  1.000000e+00
```

Is the Cholesky factor L a matrix square root of S in the sense that $L^2 = S$. Explain carefully.

```
print(L)
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.9980754 0.0000000 0.0000000 0.0000000 0.0000000
## [2,] 1.1529953 1.5948862 0.0000000 0.0000000 0.0000000
## [3,] 0.1791218 0.2242364 0.4030376 0.0000000 0.0000000
## [4,] 0.2854546 0.4181449 0.7927371 1.2132447 0.0000000
## [5,] 0.0817068 0.1893991 0.2923464 0.4121685 0.8087086
```

```
max(abs(L %*% L - S))
```

```
## [1] 1.838896
```

```
max(abs(L %*% t(L) - S))
```

```
## [1] 1.110223e-16
```

Since L is not a symmetric matrix, but lower triangular it cannot be directly seen as a square root. But still the values are, because if you do the transposition you still get the view that this is the square root.

5. Compare the three whitened data sets.

- Are the transformed observations identical?

Not at all, the differences are on the scale of the data itself (not like any precision error of e-16 or the like)

- Do they have the same covariance matrix?

Yep, this they do. The differences are all at the scale of precision (e-15)

- Which aspects of the data have changed differently?

So generally this whitening is a scaling of the original data based on the correlation of its variables. This scaling removes the correlations between the variables and scales the variance to 1 (as we have seen now in all three steps with cov and the 1 diagonal matrix)!

The three different types then apply it differently, so I would say the transformation is the same in the sense of covariance but the coordinate system this is transformed it is then different. So how the new variables relate to the old in a way.

We see that the biggest correlation between the transformed and old data is for the ZCA, which is using a matrix that I before described as a scaled covariance matrix S of the data (with ^-1/2). So this data probably stays closer to the original, making it still interpretable.

```
Z_pca  <- Xc %*% t(W_PCA)
Z_zca  <- Xc %*% t(W_ZCA)
Z_chol <- Xc %*% t(L_inv)

print("Transformed Observations")
```

```
## [1] "Transformed Observations"
```

```
head(round(Xc, 3), 3)
```

```
##        [,1]   [,2]   [,3]   [,4]   [,5]
## [1,] -0.968 -0.422 -0.448  0.906  1.054
## [2,] -0.299 -2.148 -0.239 -1.956 -0.947
## [3,]  0.252 -1.221  0.057 -0.846 -1.758
```

```
head(round(Z_pca, 3), 3)
```

```
##       [,1]   [,2]   [,3]   [,4]   [,5]
## [1,] 0.048 -0.989 -1.156 -0.197 -1.576
## [2,] 1.302  0.616  0.451 -0.150  0.882
## [3,] 0.722  0.576  1.474  1.081  0.927
```

```
head(round(Z_zca, 3), 3)
```

```
##        [,1]   [,2]   [,3]   [,4]   [,5]
## [1,] -1.023  0.013 -1.518  0.645  1.036
## [2,]  0.180 -1.053  0.615 -1.118 -0.555
## [3,]  0.568 -0.733  0.851 -0.168 -1.855
```

```
head(round(Z_chol, 3), 3)
```

```
##        [,1]   [,2]   [,3]   [,4]   [,5]
## [1,] -0.970  0.437 -0.924  1.428  0.905
## [2,] -0.299 -1.131  0.170 -1.263 -0.294
## [3,]  0.253 -0.948  0.558 -0.794 -1.774
```

```
print("Transformed Observations - Max diffs")
```

```
## [1] "Transformed Observations - Max diffs"
```

```
c(pca_vs_zca  = max(abs(Z_pca - Z_zca)),
  pca_vs_chol = max(abs(Z_pca - Z_chol)),
  zca_vs_chol = max(abs(Z_zca - Z_chol)))
```

```
##  pca_vs_zca pca_vs_chol zca_vs_chol
##    5.464319    6.089656    1.711604
```

```
print("Comparing the covs")
```

```
## [1] "Comparing the covs"
```

```
c(pca  = max(abs(cov(Z_pca)  - diag(5))),
  zca  = max(abs(cov(Z_zca)  - diag(5))),
  chol = max(abs(cov(Z_chol) - diag(5))))
```

```
##          pca          zca         chol
## 2.886580e-15 3.108624e-15 2.442491e-15
```

```
print("How correlated are the original data and the transformed one?")
```

```
## [1] "How correlated are the original data and the transformed one?"
```

```
round(cor(Xc, Z_pca), 2)
```

```
##       [,1]  [,2]  [,3]  [,4]  [,5]
## [1,] -0.63  0.30  0.63 -0.36 -0.01
## [2,] -0.93  0.33 -0.12  0.06 -0.02
## [3,] -0.71 -0.23  0.03  0.09  0.66
## [4,] -0.62 -0.75  0.13  0.17 -0.03
## [5,] -0.42 -0.59 -0.35 -0.60  0.00
```

```
round(cor(Xc, Z_zca), 2)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,] 0.91 0.40 0.06 0.07 0.01
## [2,] 0.20 0.96 0.11 0.13 0.05
## [3,] 0.13 0.42 0.78 0.42 0.13
## [4,] 0.05 0.17 0.14 0.95 0.22
```

```
## [5,] 0.01 0.11 0.07 0.34 0.93
```

```
round(cor(Xc, Z_chol), 2)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,] 1.00 0.00 0.00 0.00 0.00
## [2,] 0.59 0.81 0.00 0.00 0.00
## [3,] 0.36 0.45 0.81 0.00 0.00
## [4,] 0.19 0.27 0.52 0.79 0.00
## [5,] 0.08 0.19 0.30 0.42 0.83
```

```
print("Maybe the diagonals still?")
```

```
## [1] "Maybe the diagonals still?"
```

```
round(rbind(pca  = diag(cor(Xc, Z_pca)),
            zca  = diag(cor(Xc, Z_zca)),
            chol = diag(cor(Xc, Z_chol))), 3)
```

```
##        [,1]  [,2]  [,3]  [,4]  [,5]
## pca  -0.625 0.330 0.034 0.167 0.000
## zca   0.912 0.963 0.783 0.950 0.931
## chol  1.000 0.810 0.815 0.790 0.829
```

```
print("Checking the strucutre of the whitening matrices" )
```

```
## [1] "Checking the strucutre of the whitening matrices"
```

```
round(W_PCA, 3)
```

```
##        [,1]   [,2]   [,3]   [,4]   [,5]
## [1,] -0.125 -0.369 -0.071 -0.192 -0.082
## [2,]  0.136  0.298 -0.051 -0.529 -0.262
## [3,]  1.040 -0.399  0.028  0.319 -0.564
## [4,] -0.647  0.198  0.082  0.464 -1.066
## [5,] -0.052 -0.303  2.962 -0.471 -0.001
```

```
round(W_ZCA, 3)
```

```
##        [,1]   [,2]   [,3]   [,4]   [,5]
## [1,]  1.212 -0.249 -0.059 -0.010  0.027
## [2,] -0.249  0.619 -0.259 -0.057 -0.029
## [3,] -0.059 -0.259  2.931 -0.357 -0.046
## [4,] -0.010 -0.057 -0.357  0.809 -0.267
## [5,]  0.027 -0.029 -0.046 -0.267  1.206
```

```
round(L_inv, 3)
```

```
##        [,1]   [,2]   [,3]   [,4]  [,5]
## [1,]  1.002  0.000  0.000  0.000 0.000
## [2,] -0.724  0.627  0.000  0.000 0.000
## [3,] -0.042 -0.349  2.481  0.000 0.000
## [4,]  0.042  0.012 -1.621  0.824 0.000
## [5,]  0.063 -0.027 -0.071 -0.420 1.237
```

6. Show mathematically that if W is a whitening matrix and R is any orthogonal matrix, then RW is also a whitening matrix.

Definition:

A matrix W is a whitening matrix if the transformed observations

$$Z = XW^T$$

have identity covariance:

$$Cov(Z) = WSW^T = I_p$$

Then:

$$Cov(X(RW)^T) = RWS(RW)^T = RWSW^TR^T = RI_pR^T = RR^T = I_p$$

R here is essentially doing the same as PCA in the beginning where we effectively did a rotation/reflection with the orthogonal Q matrix (as R is also orthogonal). Applying R only rotates/reflects the already whitened data, and since the identity covariance is invariant under orthogonal transformations ($RIR^T = I$), the result stays white, producing another non-unique result (although in the covariance we get the identity covariance which is the design goal anyhow). There are unlimited results, as there are infinitely orthogonal matrices.

7. More difficult dataset

Re-centre the data and investigate:

- the eigenvalues of S;
  - The fifth eigenvalue is e-15 which is expected since it is almost a linear combination of the first one, the others are perfectly okay and close to the previous order of magnitude.
- kappa(S);
  - So the kappas are all e15, so they point to a very ill-conditioned problem. Meaning small perturbations in the data (so also rounding errors!) can lead to larger changes in the solution.
- the three whitening transformations;
  - All three whitening matrices contain very large entries because there is some inversion of S involved (for L, you have one time 9.81* 10^-8 which translates into order 10°7 for the inverse)
- the covariance matrices after whitening.
  - The "quality" has deteriorated. They are quite far off from fulfilling the identity covariance $Cov(Z) = I_p$
  - This is all coming from the introduction of the correlation between 1 and 5 as one can see quite well in the covariance matrices at the end where almost all diagonal entries are 1

    What numerical difficulties do you observe?
  - S is really close to being singular, but not yet. But this still amplifies the values of the Z matrices, making it really volatile for small perturbations in the input (rounding errors as well as already mentioned)
  - I did not check for errors in the code explicitly, so it could break in case we really get close to singularity or rather a small perturbations pushes us there.

```
# lets keep the original X separate if we go up again
X_bad <- X
X_bad[, 5] <- X_bad[, 1] + 1e-7 * rnorm(n)
```

```r
Xc_bad <- scale(X_bad, center = TRUE, scale = FALSE)
S_bad <- crossprod(Xc_bad) / (n - 1)

eig_bad <- eigen(S_bad, symmetric = TRUE)
lambdas_bad <- eig_bad$values
Qs_bad <- eig_bad$vectors

lambdas_bad
```

```
## [1] 5.317946e+00 2.007023e+00 1.031912e+00 1.100183e-01 3.759251e-15
```

```r
lambdas
```

```
## [1] 4.9784840 2.1811237 0.6020848 0.5509560 0.1100034
```

```r
c(kappa_est   = kappa(S_bad),
  kappa_exact = kappa(S_bad, exact = TRUE),
  ratio       = max(lambdas_bad) / min(lambdas_bad),
  inv_eps     = 1 / .Machine$double.eps)
```

```
##    kappa_est  kappa_exact        ratio      inv_eps
## 2.327292e+15 1.489200e+15 1.414629e+15 4.503600e+15
```

```r
lambdas_bad^(-1/2)
```

```
## [1] 4.336387e-01 7.058686e-01 9.844160e-01 3.014862e+00 1.630983e+07
```

```r
# PCA
W_PCA_bad <- diag(lambdas_bad^(-1/2)) %*% t(Qs_bad)
Z_pca_bad <- Xc_bad %*% t(W_PCA_bad)

# ZCA
W_ZCA_bad <- Qs_bad %*% diag(lambdas_bad^(-1/2)) %*% t(Qs_bad)
Z_zca_bad <- Xc_bad %*% t(W_ZCA_bad)

# Cholesky
L_bad <- t(chol(S_bad))
L_inv_bad  <- forwardsolve(L_bad, diag(5))
Z_chol_bad <- Xc_bad %*% t(L_inv_bad)
diag(L_bad)
```

```
## [1] 9.980754e-01 1.594886e+00 4.030376e-01 1.213245e+00 8.493505e-08
```

```r
# there are null values
dev_I <- function(M) max(abs(M - diag(5)))

rbind(
  WSWt  = c(pca  = dev_I(W_PCA_bad %*% S_bad %*% t(W_PCA_bad)),
            zca  = dev_I(W_ZCA_bad %*% S_bad %*% t(W_ZCA_bad)),
            chol = dev_I(L_inv_bad %*% S_bad %*% t(L_inv_bad))),
  cov_Z = c(pca  = dev_I(cov(Z_pca_bad)),
            zca  = dev_I(cov(Z_zca_bad)),
            chol = dev_I(cov(Z_chol_bad)))
)
```

```
##              pca        zca       chol
## WSWt  0.07592322 0.02951757 0.02346049
## cov_Z 0.29946031 0.14973017 0.35431759
```

```r
# same checks for the "good" X from before
rbind(
  good = c(pca  = dev_I(cov(Z_pca)),
           zca  = dev_I(cov(Z_zca)),
           chol = dev_I(cov(Z_chol)))
)
```

```
##              pca          zca         chol
## good 2.88658e-15 3.108624e-15 2.442491e-15
```

```r
c(pca  = max(abs(W_PCA_bad)),
  zca  = max(abs(W_ZCA_bad)),
  chol = max(abs(L_inv_bad)))
```

```
##      pca      zca     chol
## 11532788  8154913 11773703
```

```r
round(cov(Z_pca_bad), 3)
```

```
##      [,1] [,2] [,3] [,4]  [,5]
## [1,]    1    0    0    0 0.000
## [2,]    0    1    0    0 0.000
## [3,]    0    0    1    0 0.000
## [4,]    0    0    0    1 0.000
## [5,]    0    0    0    0 1.299
```

```r
round(cov(Z_zca_bad), 3)
```

```
##       [,1] [,2] [,3] [,4]  [,5]
## [1,]  1.15    0    0    0 -0.15
## [2,]  0.00    1    0    0  0.00
## [3,]  0.00    0    1    0  0.00
## [4,]  0.00    0    0    1  0.00
## [5,] -0.15    0    0    0  1.15
```

```r
round(cov(Z_chol_bad), 3)
```

```
##      [,1] [,2] [,3] [,4]  [,5]
## [1,]    1    0    0    0 0.000
## [2,]    0    1    0    0 0.000
## [3,]    0    0    1    0 0.000
## [4,]    0    0    0    1 0.000
## [5,]    0    0    0    0 1.354
```

8. Suppose one eigenvalue of S is numerically zero. Suggest at least two reasonable ways to proceed if whitening is still required. Discuss both the numerical and statistical consequences.

1. **Dimensionality reduction:**

   Keep only dimensions of PCA/ZCA (so truncating) where the lambda/eigenvalue is clearly above 0 (e.g. by defining a threshold) and only use those for the whitening process

   1. Numerical consequence: The blown scaling factor of around 10^7 that we saw before will not be present anymore. Kappa hence would point to a well-conditioned issue.

2. Statistical consequence: Since we removed at least one dimension Cov(Z) should only be I_r (with r<p) anymore. With ZCA in the last example we would lose even two columns (as the influence was reflected in both col 1 and 5!)

2. **Removing a variable in the beginning:**

   Drop the near linear dependent variables from the beginning (i.e. here X_5)

   1. Numerical consequence: Similar to before.
   2. Statistical consequence: The identity covariance holds (also with reduced dimension ofc). Since we only remove "near linear dependent" variables we have almost no information lost since e.g. $X_5 \approx X_1$. Before we remove a linear combination, here really already the original variable, making it probably a bit more interpretable.